

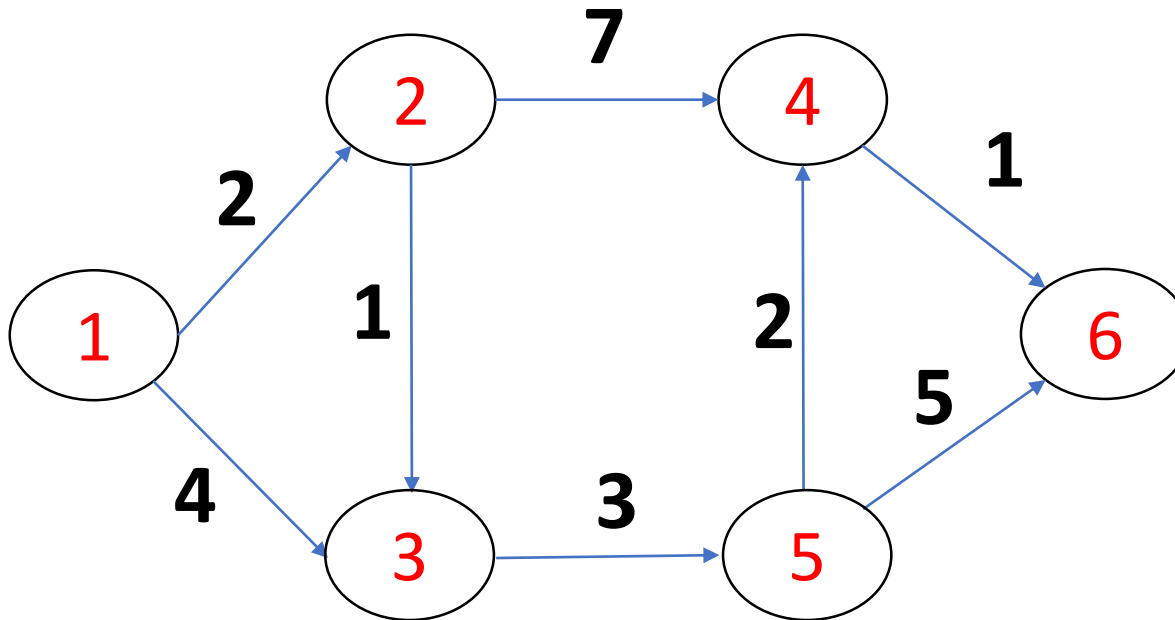
Graph Algorithms

MODULE V

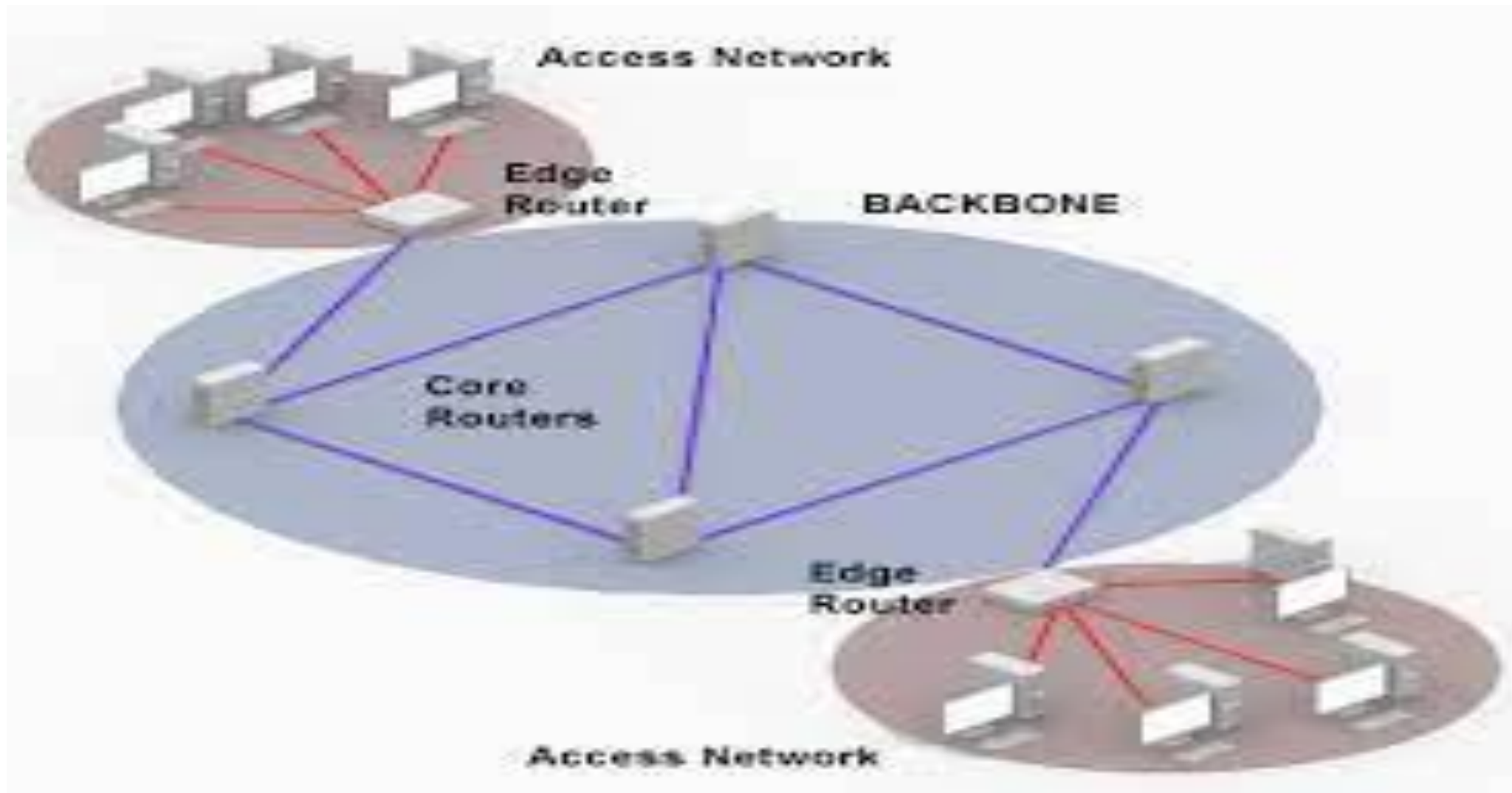
Single source shortest path problem using Greedy method

Shortest path algorithms

- Single source shortest path algorithm [Dijkstra's Algorithm]
- All pairs shortest path algorithm [Floyd – Warshall Algorithm]



One application of Dijkstra's Algorithm



Conditions

- Dijkstra algorithm works only for connected graphs.
- Dijkstra algorithm works only for those graphs that do not contain any negative weight edge.
- The actual Dijkstra algorithm does not output the shortest paths.
- It only provides the value or cost of the shortest paths.
- By making minor modifications in the actual algorithm, the shortest paths can be easily obtained.
- Dijkstra algorithm works for directed as well as undirected graphs.

1. $\text{dist}[S] \leftarrow 0$ // The distance to source vertex is set to 0.
2. $\Pi[S] \leftarrow \text{NIL}$ // The predecessor of source vertex is set as NIL.
3. for all $v \in V - \{S\}$ // for all other vertices
4. Do
5. $\Pi[v] \leftarrow \text{NIL}$ //The predecessor of all other vertices is set as NIL
6. $S \leftarrow \emptyset$ // The set of vertices that have been visited 'S' is initially empty.
7. $Q \leftarrow V$ //The queue 'Q' initially holds all vertices.
8. while $Q \neq \emptyset$ //While executes till queue is not empty
9. do
10. $u \leftarrow \text{mindistance}(Q, \text{dist})$
11. // A vertex from Q with the least distance is selected
12. $S \leftarrow S \cup \{u\}$ // Vertex 'u' is added to 'S' list
of vertices that have been visited

Dijkstra's Algorithm

13. for all $v \in \text{neighbors}[u]$ // For all the neighboring vertices of vertex 'u'
14. do if $\text{dist}[v] > \text{dist}[u] + w(u,v)$ // if any new shortest path is discovered
15. then $\text{dist}[v] \leftarrow \text{dist}[u] + w(u,v)$ //new value of shortest path is selected
16. return dist

Step-01

- In the first step. two sets are defined-
- One set contains all those vertices which have been included in the shortest path tree.
- In the beginning, this set is empty.
- Other set contains all those vertices which are still left to be included in the shortest path tree.
- In the beginning, this set contains all the vertices of the given graph.

Step-02

- For each vertex of the given graph, two variables are defined as-
- $\Pi[v]$ which denotes the predecessor of vertex 'v'
- $d[v]$ which denotes the shortest path estimate of vertex 'v' from the source vertex.
- Initially, the value of these variables is set as-
- The value of variable ' Π ' for each vertex is set to NIL i.e. $\Pi[v] = \text{NIL}$
- The value of variable ' d ' for source vertex is set to 0 i.e. $d[S] = 0$
- The value of variable ' d ' for remaining vertices is set to ∞ i.e. $d[v] = \infty$

Step-03

- The following procedure is repeated until all the vertices of the graph are processed-
- Among unprocessed vertices, a vertex with minimum value of variable 'd' is chosen.
- Its outgoing edges are relaxed.
- After relaxing the edges for that vertex, the sets created in step-01 are updated.

Time Complexity Analysis

Case 01:

- This case is valid when-
- The given graph G is represented as an adjacency matrix.
- Priority queue Q is represented as an unordered list.
- Here,
- $A[i,j]$ stores the information about edge (i,j) .
- Time taken for selecting i with the smallest dist is $O(V)$.
- For each neighbor of i , time taken for updating $\text{dist}[j]$ is $O(1)$ and there will be maximum V neighbors.
- Time taken for each iteration of the loop is $O(V)$ and one vertex is deleted from Q .
- Thus, total time complexity becomes $O(V^2)$.

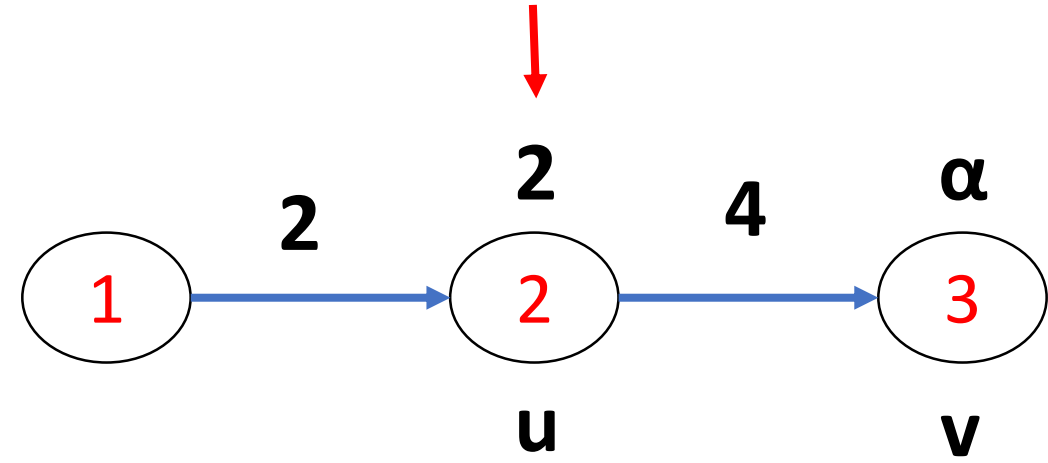
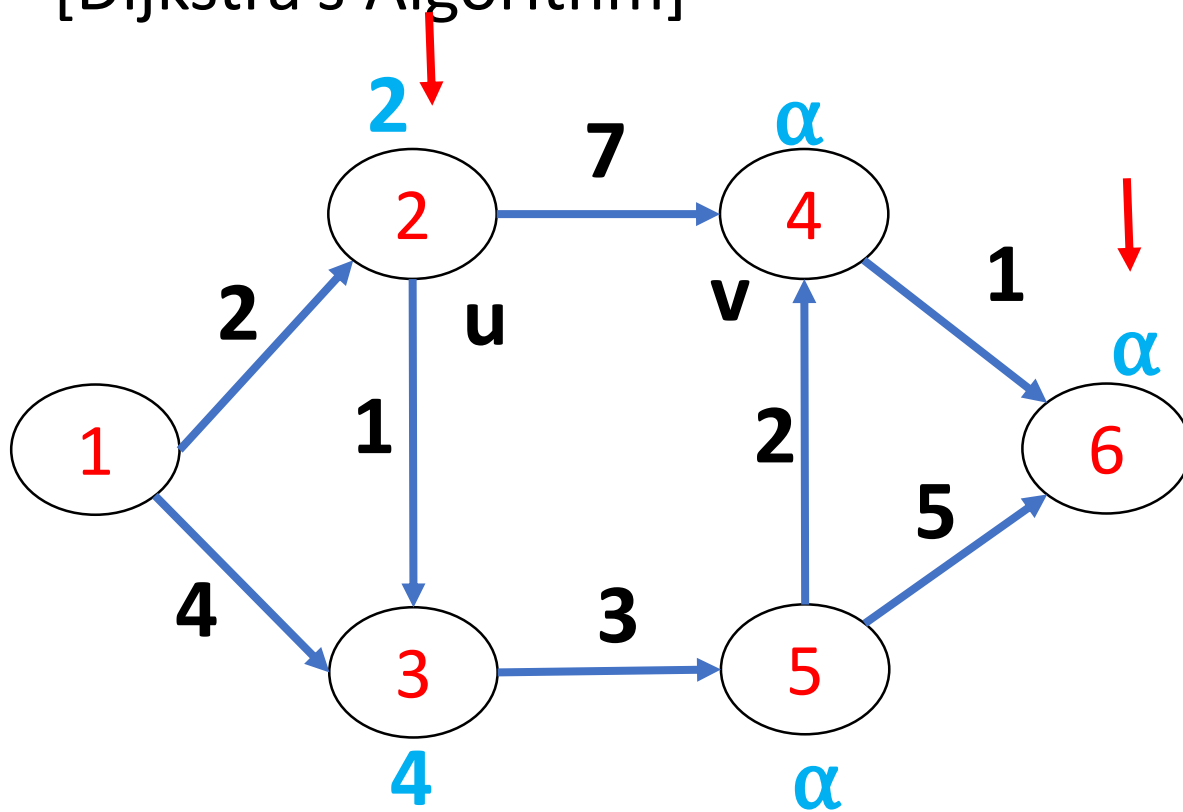
Time Complexity Analysis

Case 02:

- This case is valid when-
- The given graph G is represented as an adjacency list.
- Priority queue Q is represented as a binary heap.
- Here,
- With adjacency list representation, all vertices of the graph can be traversed using BFS in $O(V+E)$ time.
- In min heap, operations like extract-min and decrease-key value takes $O(\log V)$ time.
- So, overall time complexity becomes $O(E+V) \times O(\log V)$ which is $O((E + V) \times \log V) = O(E \log V)$
- This time complexity can be reduced to $O(E+V \log V)$ using Fibonacci heap.

Single source shortest path problem

- Single source shortest path problem [Dijkstra's Algorithm]

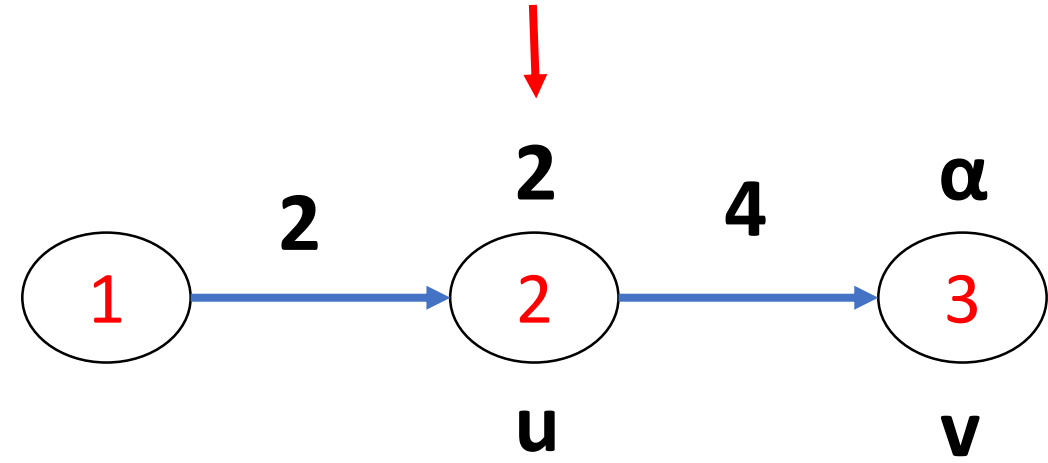
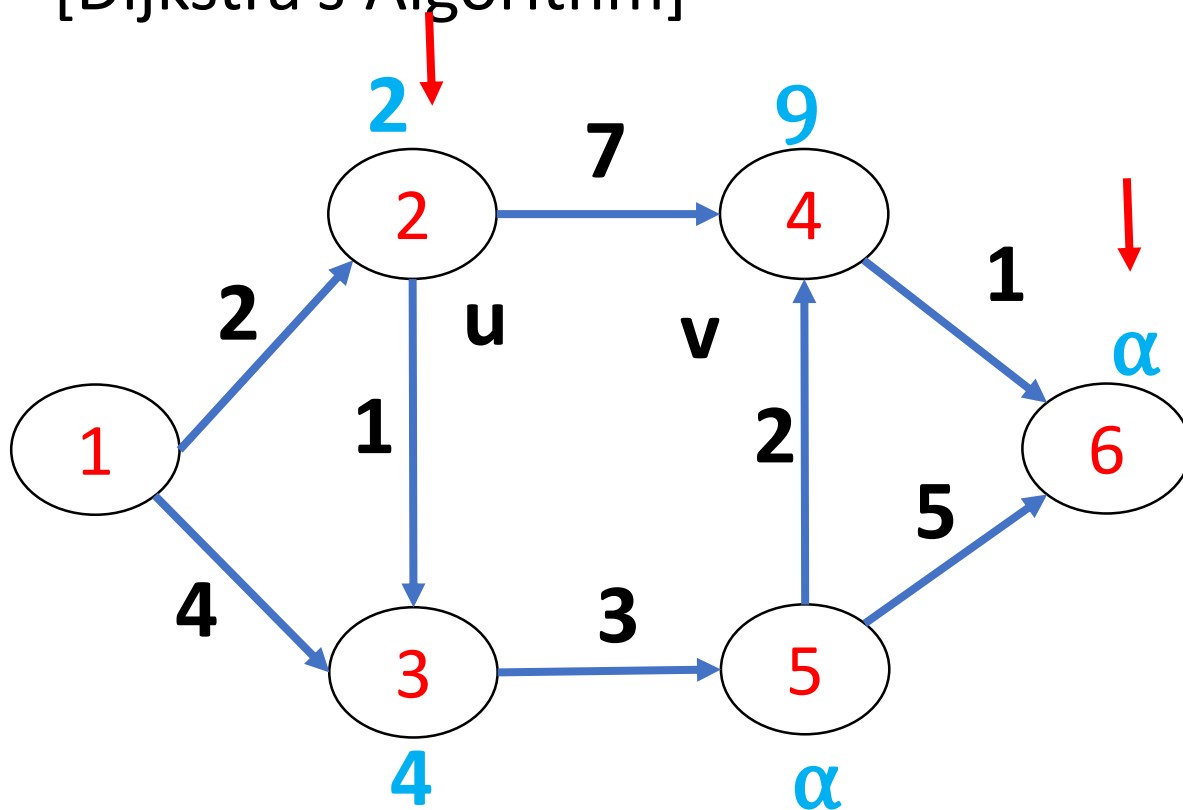


Relaxation

$$\text{If } (d[u] + c(u,v) < d[v]) \\ d[v] = d[u] + c(u,v)$$

Single source shortest path problem

- Single source shortest path problem [Dijkstra's Algorithm]

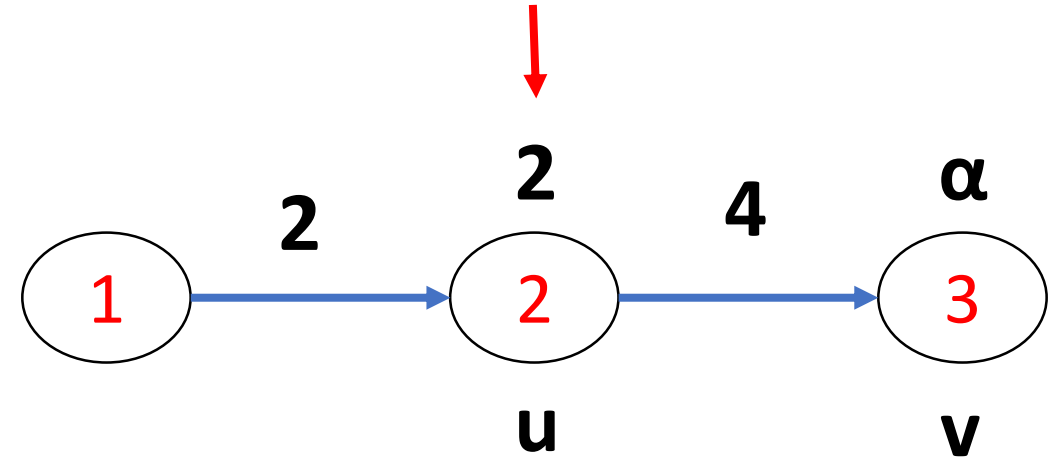
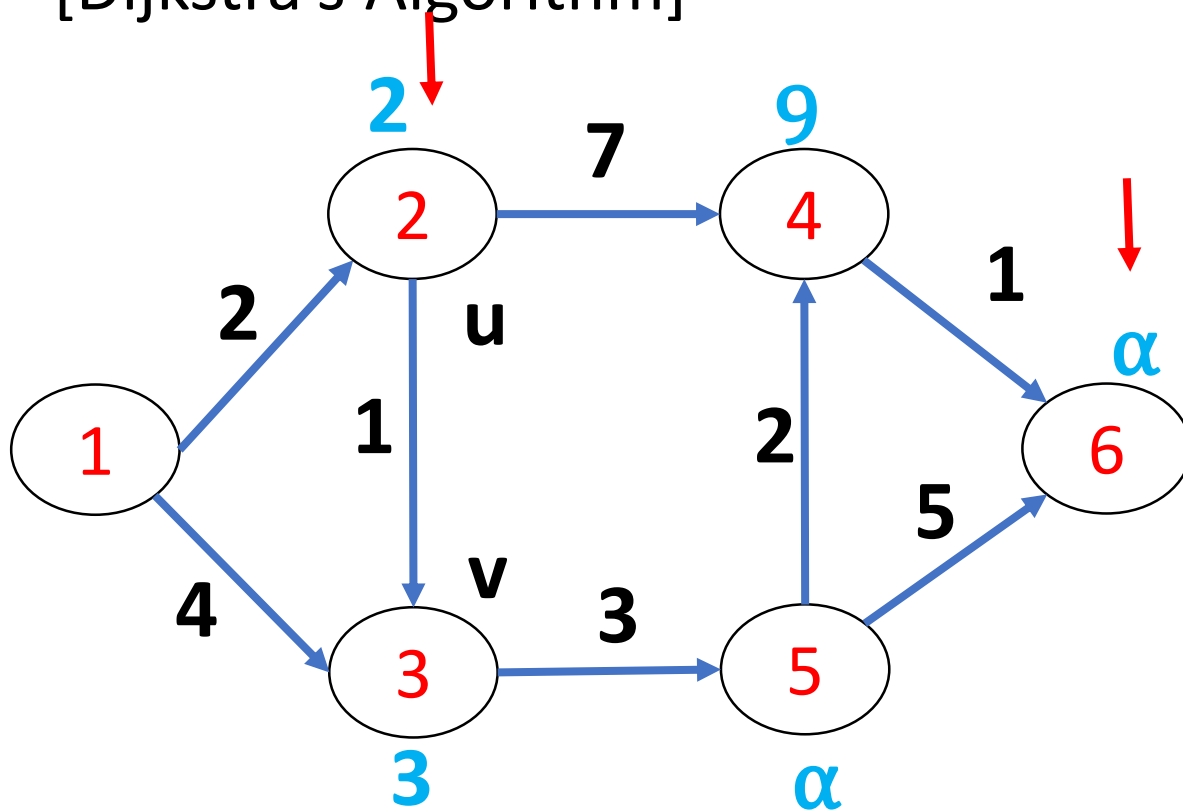


Relaxation

If ($d[u] + c(u,v) < d[v]$)
 $2 + 7 < \alpha$ **True**
 $d[v] = d[u] + c(u,v)$
 9

Single source shortest path problem

- Single source shortest path problem [Dijkstra's Algorithm]

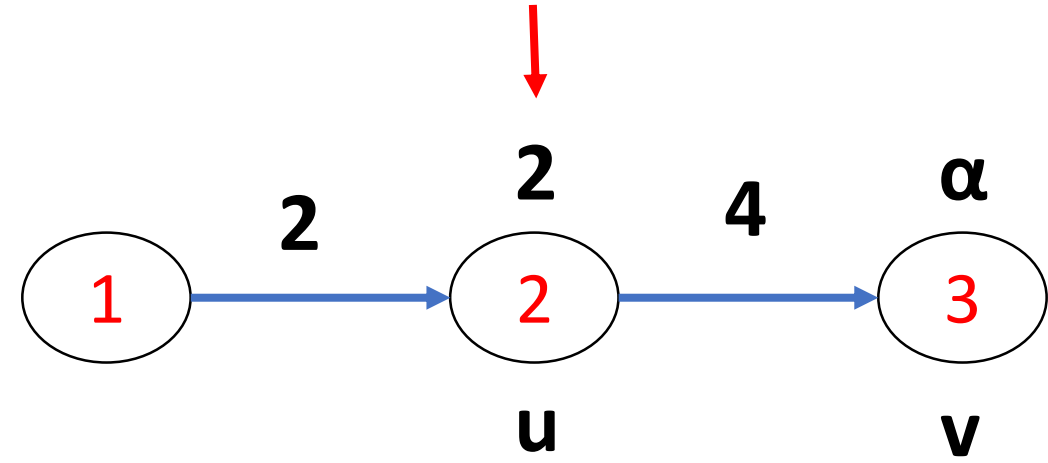
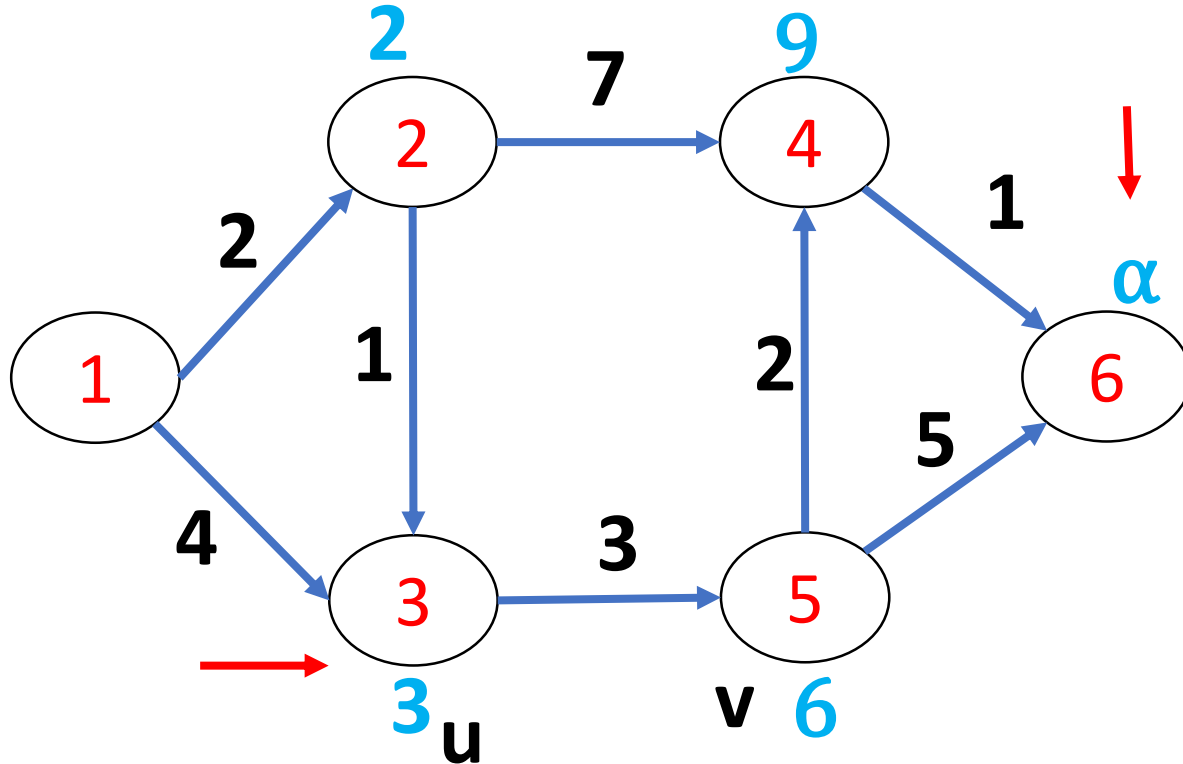


Relaxation

If ($d[u] + c(u,v) < d[v]$)
 $2 + 1 < 4$ **True**
 $d[v] = d[u] + c(u,v)$
3

Single source shortest path problem

- Single source shortest path problem [Dijkstra's Algorithm]

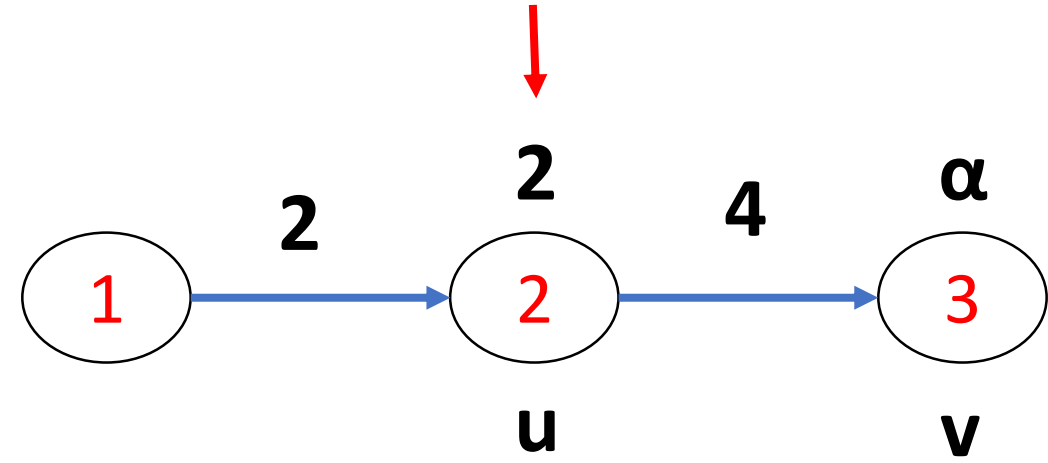
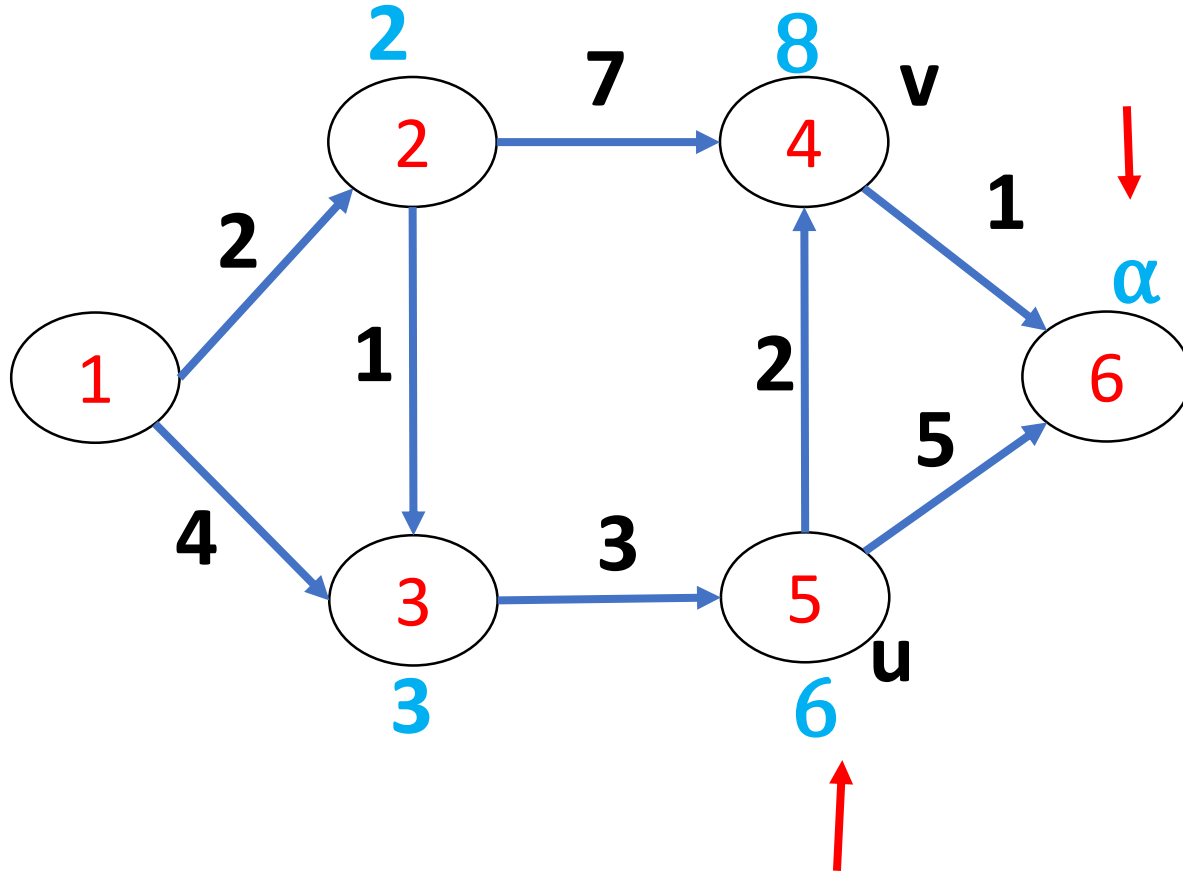


Relaxation

If ($d[u] + c(u,v) < d[v]$)
 $3 + 3 < \alpha$ True
 $d[v] = d[u] + c(u,v)$
6

Single source shortest path problem

- Single source shortest path problem [Dijkstra's Algorithm]

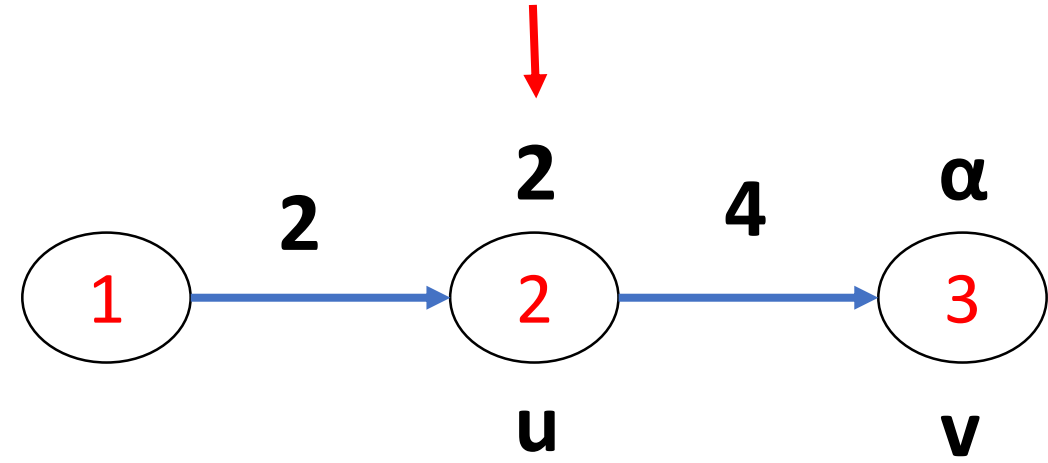
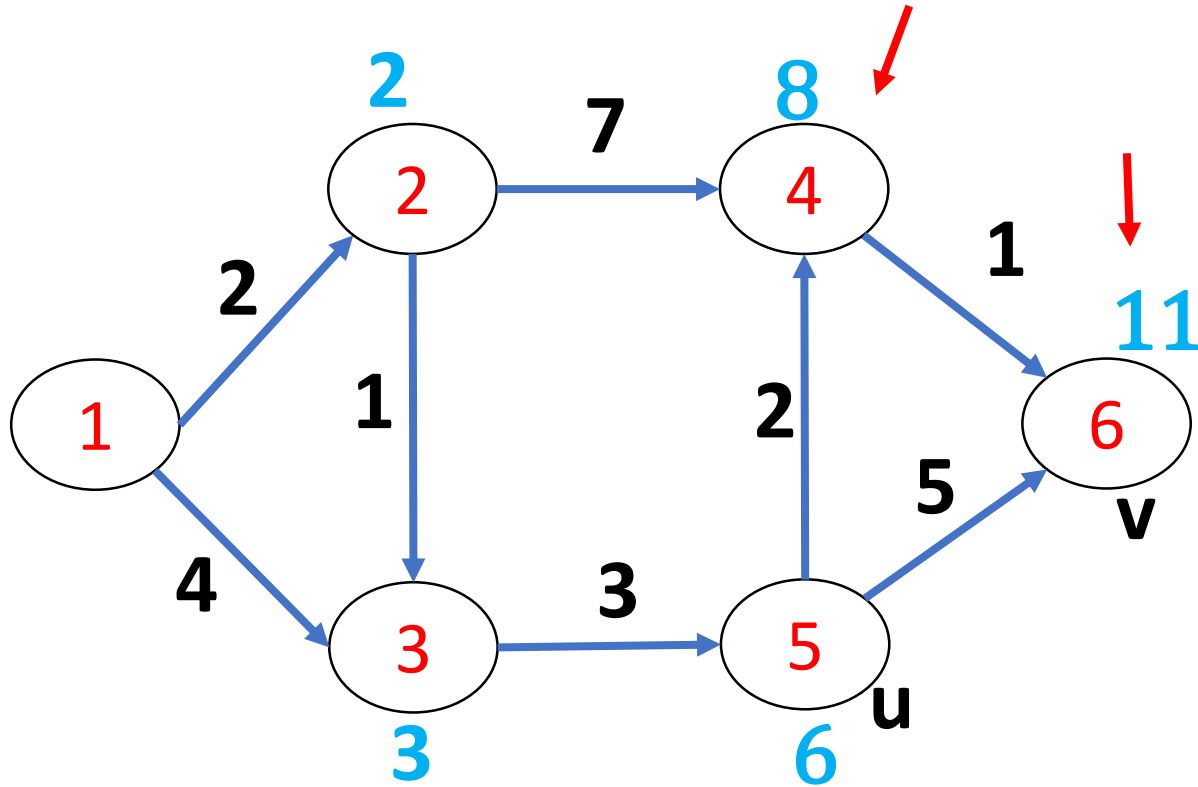


Relaxation

If ($d[u] + c(u,v) < d[v]$)
 $6 + 2 < 9$ **True**
 $d[v] = d[u] + c(u,v)$
8

Single source shortest path problem

- Single source shortest path problem [Dijkstra's Algorithm]

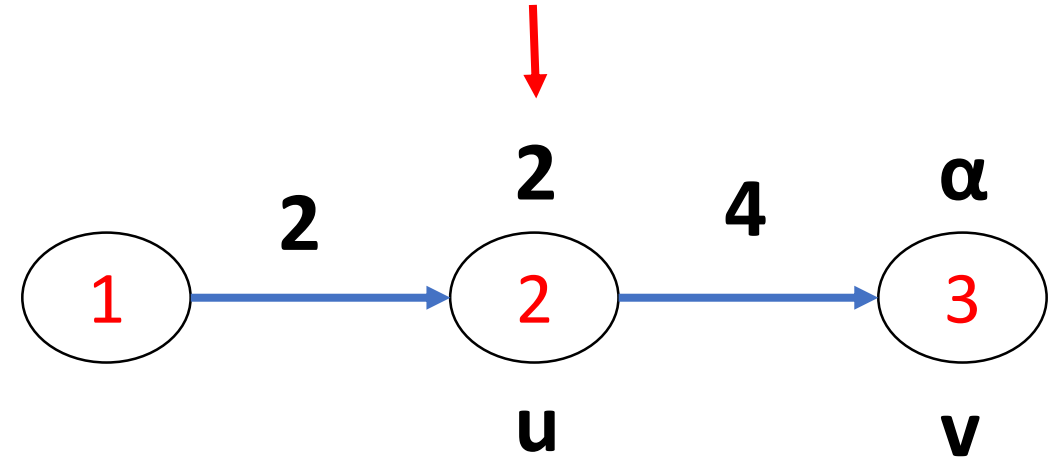
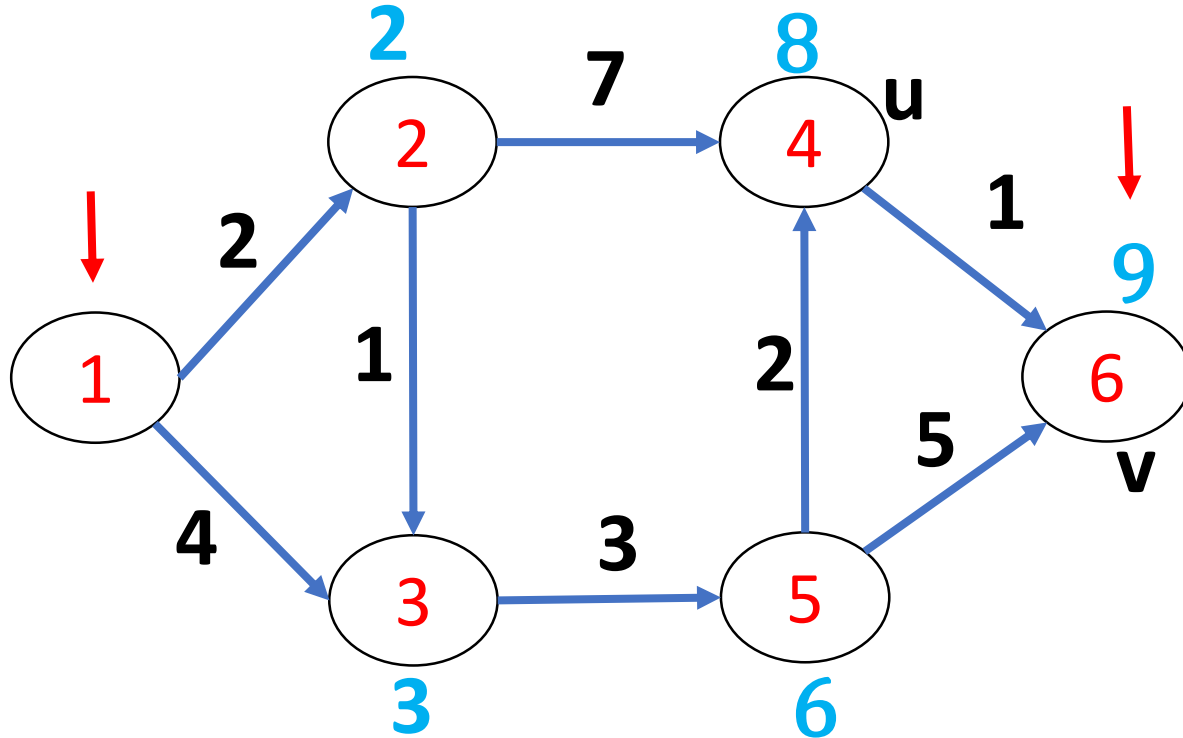


Relaxation

If ($d[u] + c(u,v) < d[v]$)
 $6 + 5 < \alpha$ True
 $d[v] = d[u] + c(u,v)$
11

Single source shortest path problem

- Single source shortest path problem [Dijkstra's Algorithm]

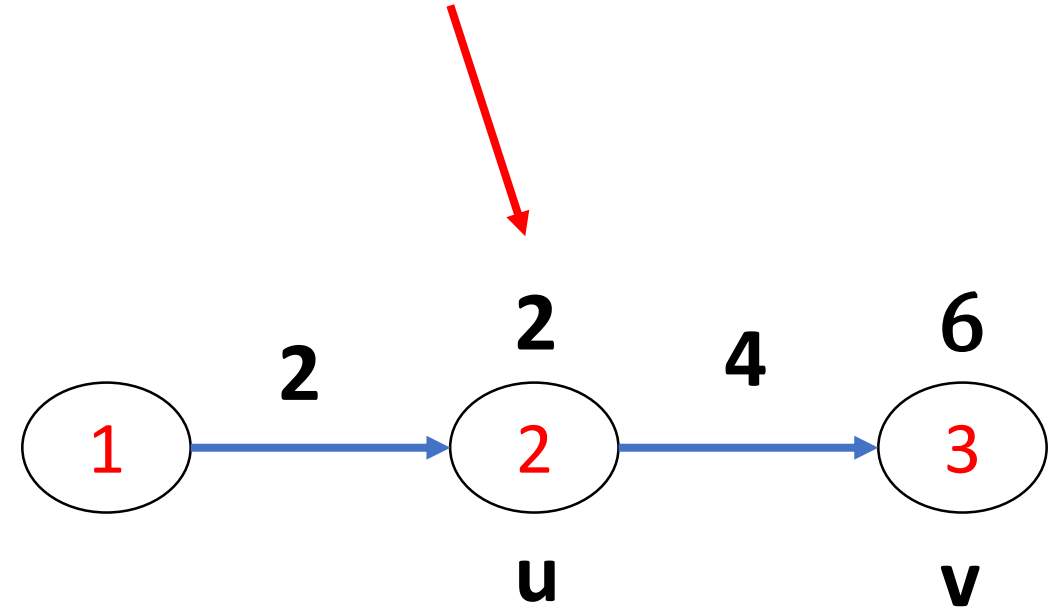
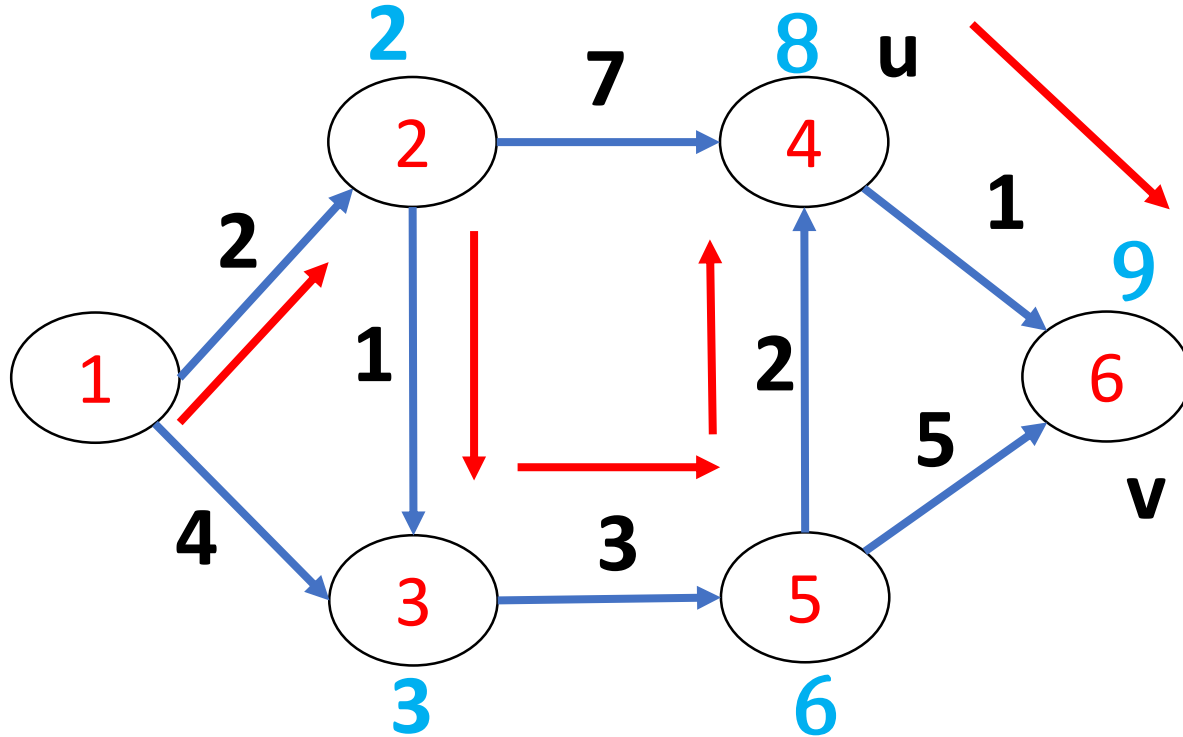


Relaxation

If ($d[u] + c(u,v) < d[v]$)
 $8 + 1 < 11$ True
 $d[v] = d[u] + c(u,v)$
9

Single source shortest path problem

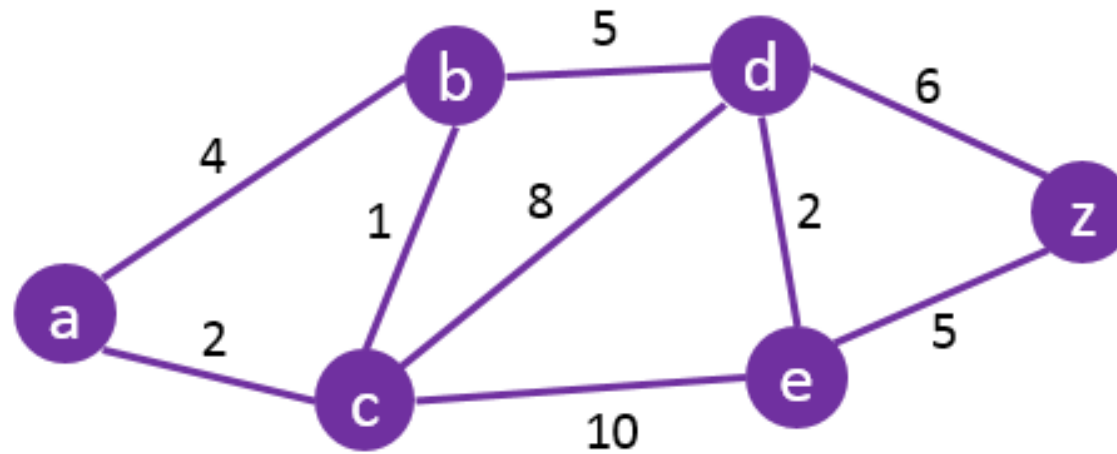
- Single source shortest path problem [Dijkstra's Algorithm]



Relaxation

If ($d[u] + c(u,v) < d[v]$)
 $d[v] = d[u] + c(u,v)$

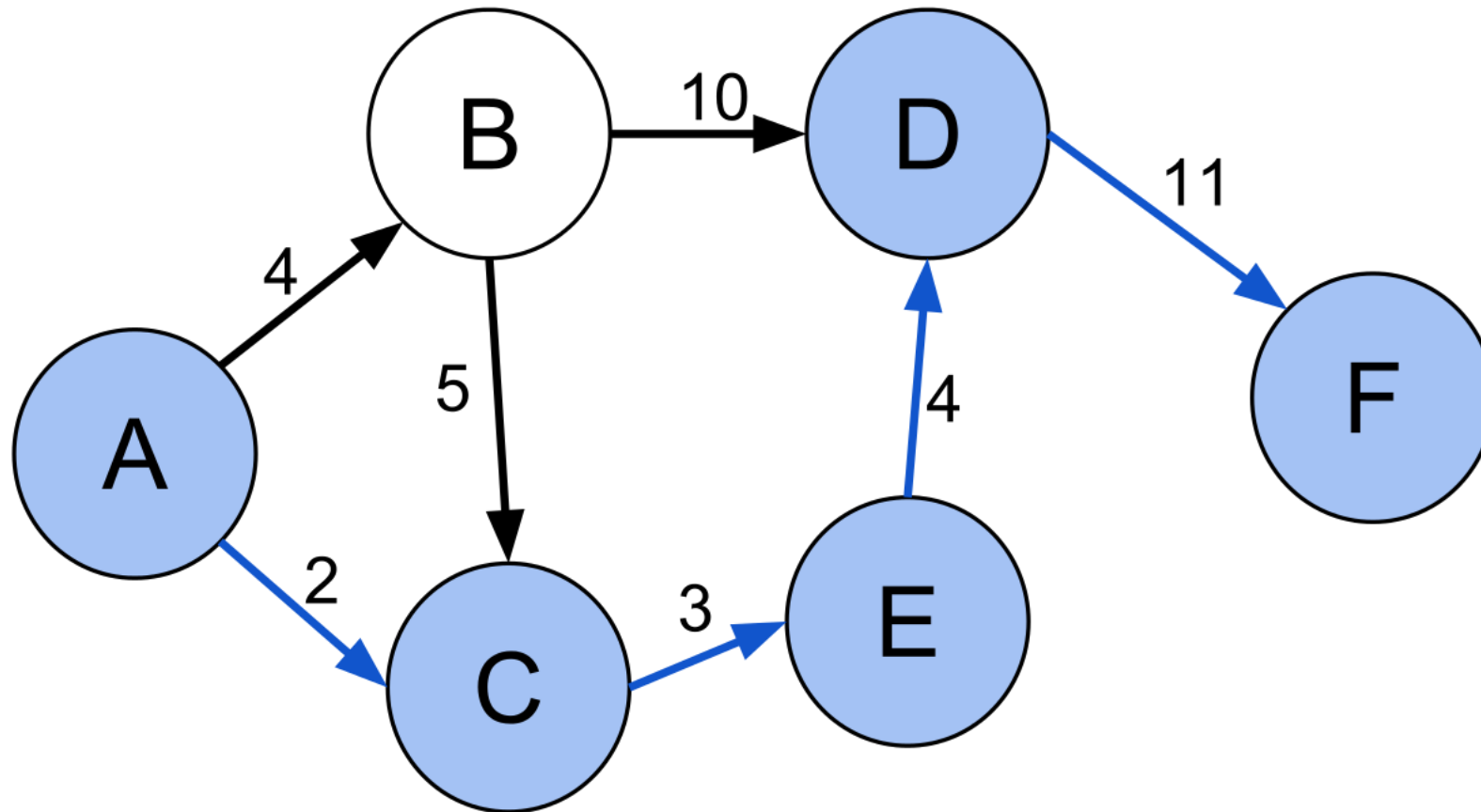
Example Problem



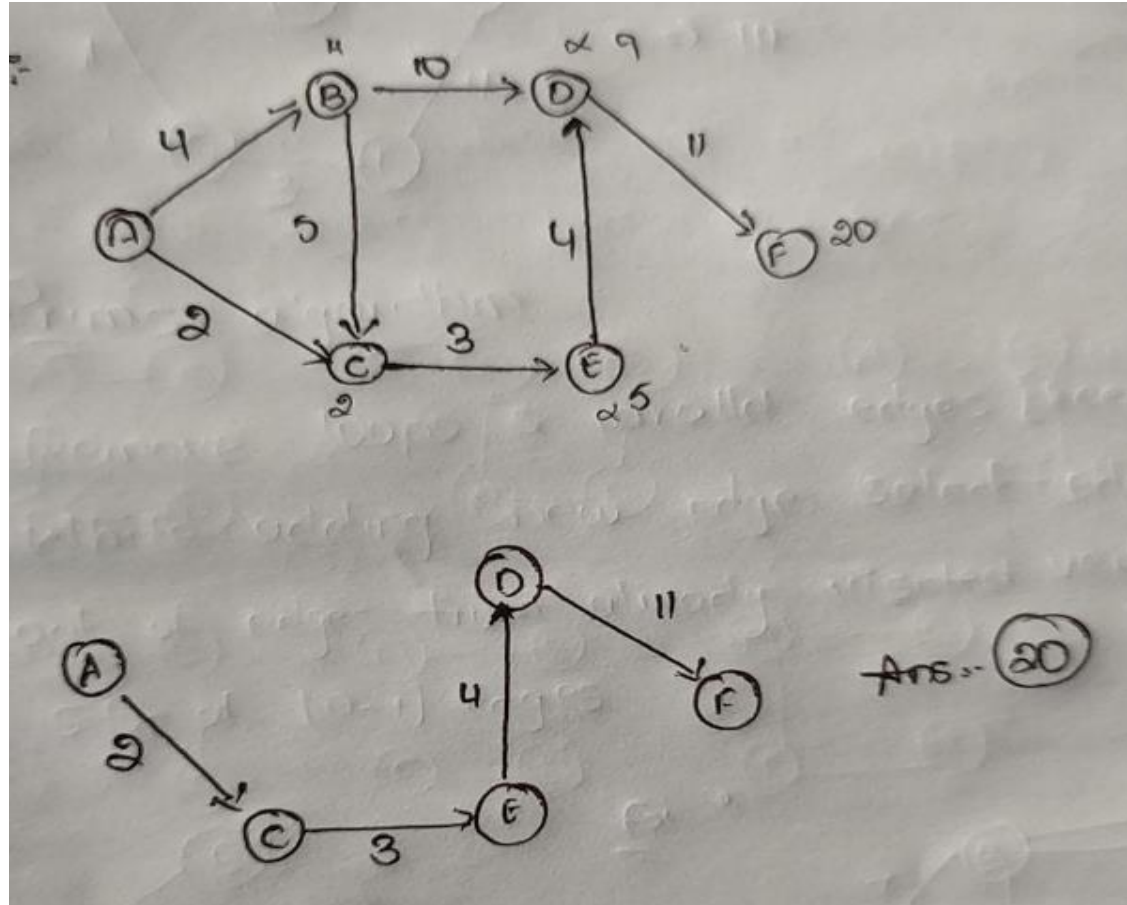
Dijkstra's Algorithm

What is the shortest path to travel from A to Z?

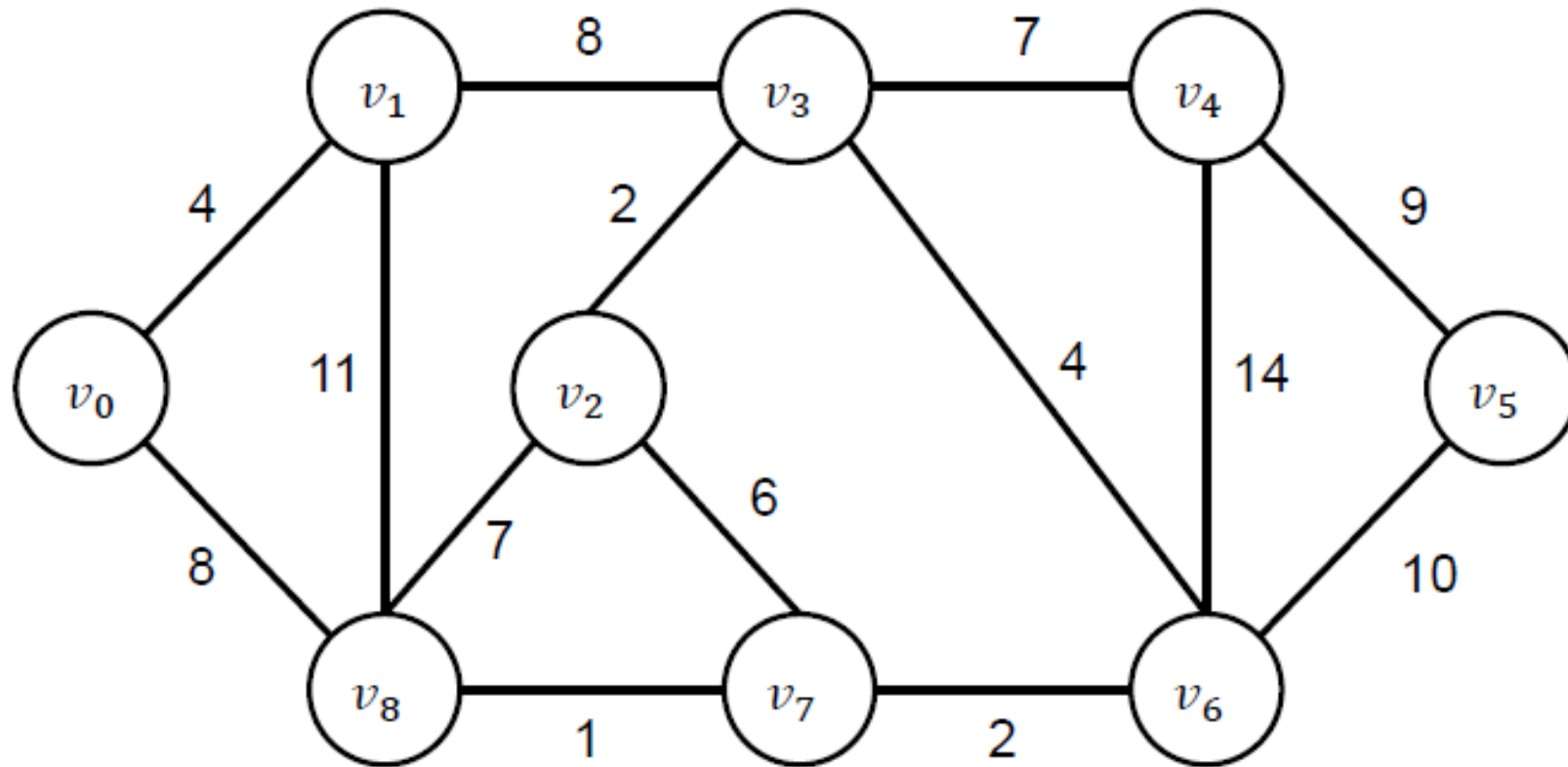
Example Problem



Example Problem



Example Problem



End

